

Comparing meta-heuristic approaches for parallel machine scheduling problems

Alexandre S. Mendes¹, Felipe M. Müller², Paulo M. França¹ and Pablo Moscato¹

¹{smendes; franca; moscato}@densis.fee.unicamp.br

²felipe@inf.ufsm.br

¹Faculdade de Engenharia Elétrica e de Computação

Universidade Estadual de Campinas - UNICAMP

C.P. 6101 CEP 13083-970 Campinas - SP - Brazil

Phone: +55 19 7883447 – Fax: +55 19 2891395

²Departamento de Eletrônica e Computação

Universidade Federal de Santa Maria

Campus Universitário CEP 97105-900 Santa Maria - RS - Brazil

Abstract: The aim of this paper is to compare the performance of two meta-heuristic methods proposed to solve the identical parallel machine scheduling problem with sequence dependent setup times. The goal is to minimize the makespan. The first algorithm is a tabu search based heuristic and the second a memetic approach, which combines a population-based method with local search procedures. As benchmarks for small-sized instances, we use optimal and lower bound values provided by a dichotomous search. For larger instances, the comparisons rely on the best known solutions provided by an exhaustive search variant of the tabu procedure.

Keywords: Machine scheduling, Parallel machines, Meta-heuristics, Tabu search, Memetic algorithms.

1. INTRODUCTION

This paper deals with the problem of scheduling a given set of n jobs to m identical parallel machines with the objective of minimizing the makespan. There is a sequence dependent machine setup time, i.e., the machine changeover time is dependent on which job is currently being processed on the machine. This made-to-stock scheduling problem arises frequently from real world settings where the setup times are significant, such as those encountered in the paper, chemical and textile industries. Following the three-field classification proposed by Graham *et al.* (1979) for scheduling problems, our problem can be categorized as $P|sds|C_{max}$.

Despite its relevance, few algorithms have been proposed to solve this problem. Dearing and Anderson (1984) developed an integer linear programming model for loom assignment in a textile weaving operation. They reported results found through the rounding of the solutions obtained by the linear relaxation of the integer model. Sumicharst and Baker (1987) propose a heuristic method based on the solution of a series of 0-1 integer subproblems that improves

the results of Dearing and Anderson. These two articles deal with a slightly different problem from ours because they assume that a job can be split among several machines. In our case the problem is combinatorially more complex. To the authors' knowledge, only two papers have reported computational results for this problem. Frederickson *et al.* (1978) present approximate algorithms derived from an equivalence between the $P|sds|C_{max}$ and the Traveling Salesman Problem (TSP). The other one is the paper of França *et al.* (1996) which uses a tabu search-based heuristic in connection with a powerful neighborhood scheme that employs the concept of local and global neighbors.

This paper considers the problem of non-preemptively scheduling n jobs, $J_1, J_2, \dots, J_n$, to m identical parallel processors, $M_1, M_2, \dots, M_m$ to minimize the completion time for the entire set of jobs (*makespan*). To each job J_j is assigned an integer and positive *processing time* p_j and an integer and positive *setup time* s_{ij} , if job J_j is executed immediately after job J_i . The setup times may be considered asymmetric, that is, $s_{ij} \neq s_{ji}$. The $P|sds|C_{max}$ is a difficult combinatorial problem proved to be *NP*-hard in a strong sense because it is equivalent to the TSP when the number of processors equals one (Baker, 1974). The main contribution of this paper is a comparison between two meta-heuristic approaches developed for its solution: the tabu search implementation of França *et al.* (1996) and a new memetic algorithm (MA) specially designed to tackle the $P|sds|C_{max}$. The MA implemented belongs to the hybrid-genetic algorithm class, which combines the strength of genetic algorithms with local search. Comparisons are performed in two ways. For small-sized instances, the two meta-heuristics are compared to optimal solutions obtained by a dichotomous search scheme (or to a lower bound when it fails). For larger instances, the benchmarks are the best-known solutions provided by a long time execution of the tabu search implementation.

2. TABU SEARCH ALGORITHM

Tabu search (TS) is a meta-heuristic that guides a local search procedure to explore the solution space beyond local optimality (Glover and Laguna, 1997). The local procedure is a search that uses an operation called *move* to define the neighborhood of any solution. TS reduces the neighborhood by classifying certain moves as forbidden or *tabu*. This mechanism prevents cycling, i.e., the indefinite execution of the same sequence of moves, and directs the search to regions not explored. The TS-based algorithm proposed in this paper is an adaptation of the method developed in França *et al.* (1996), capable of dealing with structured and non-structured instances. It is composed of three phases:

Phase 1 - An initial solution is constructed by assigning all jobs to the m processors.

Phase 2 - A local search heuristic controlled by a TS scheme is used to achieve improved solutions in terms of makespan by moving jobs between different processors.

Phase 3 - This is a final phase, in which one tries to improve the sequence obtained in the busiest processor to achieve a better makespan.

The initial solution is obtained by successively assigning each job to the machine that yields the least increase in the current makespan. Before describing the two other phases, consider the following definitions. Local neighbors of a job J_j , relative to a machine, are the closest q successors and the closest q predecessors among the jobs already assigned to this machine, where q is an input parameter. The closest successor of a job J_j means the job J_k with the least s_{jk} , and the closest predecessor is the job J_i with the least s_{ij} . Global neighbors of a job J_j are its p closest successors and its p closest predecessors among all jobs, except J_j itself. These concepts are used to constrain the search to a subspace of the search space, reducing the run time. For a better understanding of the concepts of local and global neighbors consider the following example. Suppose the matrix $[s_{ij}]$ as:

s_{ij}	0	1	2	3	4	5	6	7
0	-	32	30	54	9	42	38	42
1	0	-	63	47	25	45	7	50
2	0	21	-	76	38	60	69	56
3	0	33	26	-	54	16	44	23
4	0	58	61	37	-	45	32	46
5	0	49	49	25	13	-	45	8
6	0	23	29	18	36	27	-	51
7	0	74	82	61	25	38	54	-

Notice that an additional line s_{0j} , $j= 1,..., n$ has been introduced to deal with the setup time of the first job on each machine. Assume that $s_{i0} = 0$, $i = 1,..., n$. Suppose that there are two machines and the jobs currently assigned to them are:

$$M_1 : 0 - 2 - 7 - 5$$

$$M_2 : 0 - 3 - 1 - 4 - 6$$

The local neighborhood structure for job J_1 and $q = 2$ is then

$$\begin{aligned} M_1 \\ \text{successors} &= \{ 0, 5 \} \\ \text{predecessors} &= \{ 2, 0 \} \end{aligned}$$

$$\begin{aligned} M_2 \\ \text{successors} &= \{ 0, 6 \} \\ \text{predecessors} &= \{ 6, 0 \} \end{aligned}$$

Using the same example and supposing $p=3$, the J_1 global neighbors are:

$$\begin{aligned} \text{successors} : & \{ 0 - 6 - 4 \} \\ \text{predecessors} : & \{ 2 - 6 - 0 \} \end{aligned}$$

In our TS algorithm, a move is defined as the removal of a job J_j from the busiest machine and its insertion in another machine of the set $M(J_j) = \{M_i / M_i \neq M'_i \text{ and } M_i \text{ includes among its jobs at least one } J_j\text{'s global predecessor or successor}\}$, where M'_i is the busiest machine. The removal and insertion procedures are an adaptation of the heuristics GENI and US originally

proposed for solving the symmetric TSP (Gendreau *et al.*, 1992). This paper uses the asymmetric version of these two procedures as well as its proper adaptation to the $P|sds|C_{max}$.

During Phase 2, when executing a removal or an insertion of a job in a machine, the algorithm selects the best move with respect to the finishing time of all jobs assigned to this machine. Four different types of removal/insertion moves are considered and at each iteration the best one is chosen to be performed.

Insertion types:

Type I

Figure 1 illustrates the Type I insertion while figure 2 shows the utilized legend. Consider that the jobs in a certain iteration are arranged in, say, machine M_m and that job J_x has been previously selected for insertion. For a given orientation of the sequence of jobs, consider J_k a job in the sub-sequence from J_j to J_i and J_l a job in the sub-sequence from J_i to J_j . After insertion, job J_x will appear between jobs J_i and J_j .

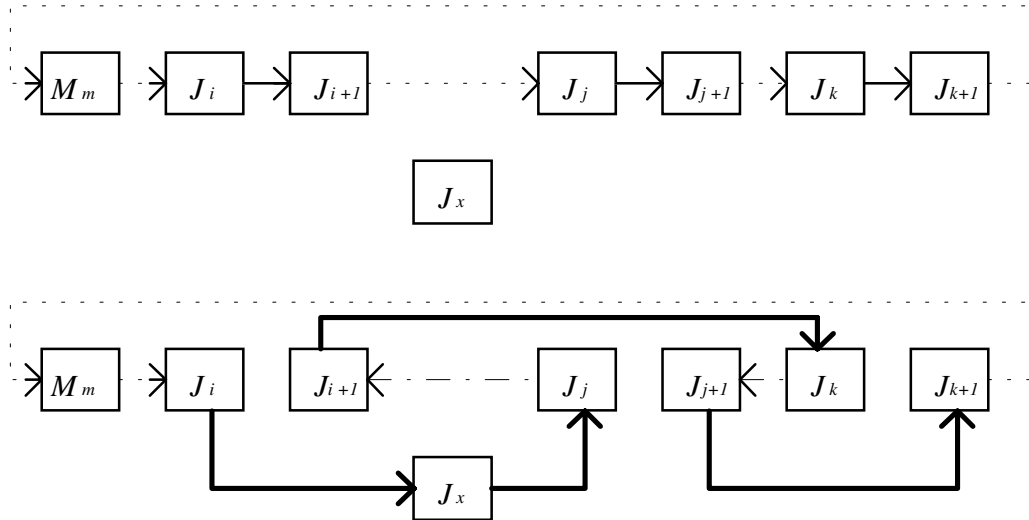


Figure 1. Example of Type I insertion

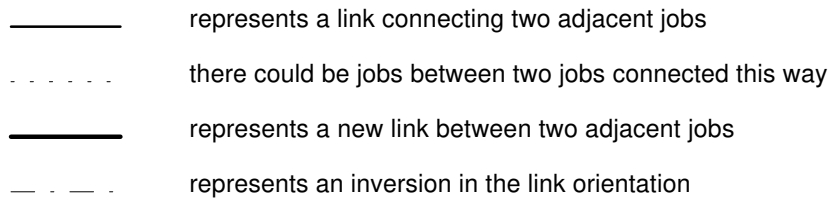


Figure 2. Legends used in figure 1

Formally, Type I insertion can be stated this way. Assume that $J_k \neq J_i$ and $J_k \neq J_j$. Inserting J_x in, say, processor M_m , results in the deletion of links (J_i, J_{i+1}) , (J_j, J_{j+1}) and (J_k, J_{k+1}) and their replacement by links (J_i, J_x) , (J_x, J_j) , (J_{i+1}, J_k) and (J_{j+1}, J_{k+1}) . Note that the sub-sequences $(J_{i+1}, \dots, J_j)$ and $(J_{j+1}, \dots, J_k)$ should appear inverted.

The other types of insertions follow.

Type II: Assuming that $J_k \neq J_j$, $J_k \neq J_{j+1}$, $J_l \neq J_i$ and $J_l \neq J_{i+1}$, the insertion of J_x in processor M_m results in the deletion of links (J_i, J_{i+1}) , (J_{l-1}, J_l) , (J_j, J_{j+1}) and (J_{k-1}, J_k) that should be replaced by links (J_i, J_x) , (J_x, J_j) , (J_l, J_{j+1}) , (J_{k-1}, J_{l-1}) and (J_{i+1}, J_k) . As a consequence, the sub-sequences $(J_{i+1}, \dots, J_{l-1})$ and $(J_l, \dots, J_j)$ are inverted.

To cope with the asymmetric aspect inherent in the $P|sds|C_{max}$, two other types of insertions can be defined.

Type III: This type of insertion can be seen as the inverse of Type I insertion. When job J_x is inserted between J_i and J_j , the sequence of jobs is rearranged in such way that almost the whole sequence is inverted. The aim is to explore other regions of the search space, hoping that valleys of new promising solutions will be found. As in Type I insertion, we assume that $J_k \neq J_i$, $J_k \neq J_j$. The insertion of J_x implies the deletion of links (J_{i-1}, J_i) , (J_{j-1}, J_j) and (J_{k-1}, J_k) that should be replaced by links (J_i, J_x) , (J_x, J_j) , (J_k, J_{j-1}) and (J_{k-1}, J_{i-1}) . This causes the inversion of the sequences $(J_i, \dots, J_{j-1})$ and $(J_k, \dots, J_{i-1})$.

Type IV: Analogously, Type IV insertion is the reverse of Type II. Again it is supposed that $J_k \neq J_j$, $J_k \neq J_{j+1}$, $J_l \neq J_i$ and $J_l \neq J_{i+1}$. The links deleted are (J_{i-1}, J_i) , (J_l, J_{l+1}) , (J_{j-1}, J_j) and (J_k, J_{k+1}) , and the links added are (J_i, J_x) , (J_x, J_j) , (J_{i-1}, J_l) , (J_{l+1}, J_{k+1}) and (J_k, J_{j-1}) . The sub-sequences that are inverted as a result are $(J_i, \dots, J_l)$ and $(J_{l+1}, \dots, J_{j-1})$.

Removal types: The removal process is the reversal of the insertion process. Hence, only the Type I removal is presented. The other three types can be derived in a straightforward manner.

Type I: Consider J_j belonging to the local neighborhood of J_{i+1} and J_k belonging to the local neighborhood of J_{i-1} , with J_k being part of the sequence $(J_{i+1}, \dots, J_{j-1})$. The removal of job J_i consists of the deletion of links (J_{i-1}, J_i) , (J_i, J_{i+1}) , (J_k, J_{k+1}) and (J_j, J_{j+1}) and in the addition of links (J_{i-1}, J_k) , (J_{i+1}, J_j) and (J_{k+1}, J_{j+1}) . The sequences $(J_{i+1}, \dots, J_k)$ and $(J_{k+1}, \dots, J_j)$ become inverted.

Following a standard TS implementation, after a move (insertion *and* removal) is performed, the reverse action is prohibited for the next θ iterations. The parameter θ is a random variable uniformly distributed in the interval $[\theta', \theta'']$. We have concluded that the best choice was $\theta'=3$ and $\theta''=7$. As in most TS implementations, if the selected move is tabu but it yields a better

makespan than the incumbent, override the tabu status and perform the move. The list of q local predecessors and successors affected by the changes in the two machines involved in the movement are updated. Phase III is a final adjustment in which we try to obtain a better makespan, by attempting to perform a removal and an insertion in the busiest machine exclusively. In case the busiest machine changes, repeat Phase III. The values for the parameters p and q are empirically taken as

$$p = \lceil \alpha \sqrt{n} \rceil \quad q = \max\{3, \lceil \alpha \sqrt{m} \rceil\}$$

where α is an input parameter. Two different versions of the TS heuristic were implemented. The first is Fast Tabu and uses a fixed value of 0.6 for α and the other is LongTabu which tries to achieve better solutions allowing α to assume four distinct values, namely, 0.6, 0.8, 1.5 and 2.0. The best solution is then selected.

3. MEMETIC ALGORITHM APPROACH

Hybrid genetic approach, also known as genetic local search or memetic algorithms (MAs) (Moscato, 1989), combines the recognized strength of the population-based methods with the intensification capability of a local search. In an MA, all individuals of the population evolve solutions until they become a local minima of a certain neighborhood (or highly evolved solutions of individual search strategies), i.e., after the recombination and mutation steps, a local search is applied to the resulting solutions. A more formal introduction to MAs and polynomial merger algorithms can be found in Moscato (1999). Figure 3 shows a pseudo-code representation of a local search-based memetic algorithm.

```

procedure Local Search-based Memetic Algorithm;
  begin
    initializePopulation Pop using FirstPop();
    forEach individual  $i \in Pop$  do  $i := \text{Local-Search}(i)$ ;
    forEach individual  $i \in \text{mainPop}$  do EvaluateFitness( $i$ );
    repeat /*generation loop */
      for  $i := 1$  to #recombinations do
        selectToMerge a set  $S_{par} \subseteq Pop$ ;
         $offspring := \text{Recombine}(S_{par}, x)$ ;
        if (selectToMutate  $offspring$ ) then  $offspring := \text{Mutate}(offspring)$ ;
         $offspring := \text{Local-Search}(offspring)$ ;
        EvaluateFitness( $offspring$ );
        addInPopulation individual  $offspring$  to Pop;
      endFor;
      if (Pop has_converged) Pop := RestartPop(Pop);
    until (termination-condition = True);
  end;

```

Figure 3. Pseudo-code of a memetic algorithm

The initialization part begins at **initializePopulation** and ends just before the **repeat** command. This part is responsible for the generation, optimization and evaluation of the initial population (*Pop*). The second part includes the so-called ‘generation loop’. At each step, two parent configurations are selected for recombination and an offspring is produced and, if selected to mutate, it suffers a mutation process. The next steps are local search, evaluation and insertion of the new solution into the population. If the population is considered to have lost diversity, a mutation process is applied on all individuals except the best one. Finally, a termination condition is checked.

Population structure

In our implementation we use a hierarchically structured population organized as a complete ternary tree of individuals clustered in 4 subpopulations or clusters, as shown in figure 4. In contrast with a non-structured population it restricts crossover possibilities. Other studies have shown that the use of structured populations is more effective when compared to non-structured populations (e.g. França *et al.* 1999; Buriol *et al.* 1999).

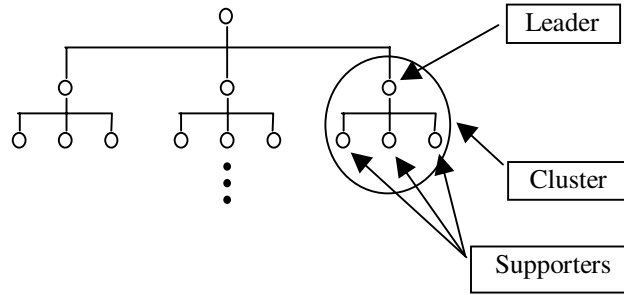


Figure 4. Population structure

The structure consists of several clusters, each one composed of a leader and three supporter solutions. The leader of a cluster is always better fitted than its supporters. This hierarchy ensures top clusters have better fitted individuals than bottom clusters. As new individuals are constantly generated, replacing old ones, periodic adjustments to keep this structure well ordered are necessary. The number of individuals in the population is restricted to the numbers of nodes in a complete ternary tree: 13, 40, 121, etc. That is, 13 individuals are necessary to construct a ternary tree with 3 levels, 40 to one with 4 levels and so on.

Representation of individuals

The representation we have chosen for the $P[sds|C_{max}]$ is quite intuitive, with a solution represented as a chromosome with the alleles assuming different integer values in the $[1, n]$ interval, where n is the number of jobs. There are $m-1$ cut-points in the chromosome that define the subsequences assigned to each machine. For instance, $\langle 3 \ 8 \ 5 \ * \ 1 \ 7 \ 4 \ 10 \ * \ 2 \ 9 \ 6 \rangle$

is a possible solution for a problem with 10 jobs and 3 machines. The cut-points (*) are in positions 4 and 9. Therefore, machine 1 executes operations 3 - 8 - 5, in this order; machine 2 executes operations 1 - 7 - 4 - 10 and machine 3 performs operations 2 - 9 - 6. As machines are identical, no distinction must be made between cut-points.

Recombination

The command **selectToMerge** indicates the task of selecting a subset of individuals (called $S_{par} \subseteq Pop$) to be used as input for the crossover operation, represented by the **Recombine()** function. In the pseudocode, the symbol 'x' stands for the instance of the problem. In this case, since we are addressing the $P[sds|C_{max}$, the 'x' refers to matrix s_{ij} and vector p_j . The crossover operator implemented is the well known Order Crossover (OX). After choosing two parents, a fragment of the chromosome from one of them is randomly selected and copied into the offspring. In the second phase, the offspring's empty positions are sequentially filled according to the chromosome of the other parent.

Parent A	<u>1</u> <u>3</u> * <u>6</u> <u>5</u> <u>2</u> * <u>7</u> <u>4</u>
Parent B	<u>5</u> <u>4</u> <u>1</u> * <u>6</u> <u>7</u> <u>3</u> * <u>2</u>
Initial Offspring	— — — <u>6</u> <u>5</u> <u>2</u> * — — (A)
Construction phase	<u>4</u> — — <u>6</u> <u>5</u> <u>2</u> * — — (B)
	<u>4</u> <u>1</u> — <u>6</u> <u>5</u> <u>2</u> * — — (B)
	<u>4</u> <u>1</u> * <u>6</u> <u>5</u> <u>2</u> * — — (B)
	<u>4</u> <u>1</u> * <u>6</u> <u>5</u> <u>2</u> * <u>7</u> — (B)
Final Offspring	<u>4</u> <u>1</u> * <u>6</u> <u>5</u> <u>2</u> * <u>7</u> <u>3</u> (B)

In the example above, the fragment is selected from the parent A and consists of the alleles $\langle 6\ 5\ 2\ * \rangle$. The child's empty positions were then filled according to the order that the alleles appear in the chromosome of parent B. The number of new individuals generated in every iteration is controlled by a parameter named *cross_rate* which is expressed as the percentage of new individuals over the total population.

Mutation

In our method, a traditional mutation strategy based on job swapping was implemented. According to it, two positions are randomly selected and the alleles in these positions swap their values. The alleles that are swapped can be both related to two jobs (two integers) or one to a job and other to a cut-point. In the first case the number of jobs on each machine remains the same. In the second case the structure of the solution is changed, because the number of jobs on each machine is modified. The case in which both positions selected are cut-points does not change anything at all.

We implemented two mutation procedures - *Mutate()* and *RestartPop()*; the first can be considered a light mutation and the other is a heavy mutation procedure. The *Mutate()* function is applied to each individual with a probability of *mut_rate* and, once applied, it mutates two alleles. Implementations with more changes per individual showed no improvement. In fact, when the number of alleles to be mutated increases, valuable information tends to be lost, worsening the MA's overall performance. The *RestartPop()* procedure, on the other hand, mutates all individuals in the *mainPop* except the incumbent solution. The swapping procedure is applied to each individual $10n$ times, so the resulting population almost resembles a randomized restarting procedure.

Fitness function

As in this problem the goal is to minimize the makespan, the fitness function was chosen as:

$$f_i = (\text{makespan}_i)^{-1}$$

where makespan_i is the makespan relative to solution i . The inversion is due to the fact that a large makespan shall be translated into a low fitness and vice-versa.

Selection of parents

Recombination is only allowed between a leader and one of its supporters and both are randomly selected. An intensification procedure was implemented, forcing the best individual to take part in approximately 10% of the crossovers. This procedure showed itself to be very effective when compared to a standard selection policy. Tests revealed small but repeated improvements over the scheme without intensification.

Offspring insertion into population

Once the leader and one supporter are selected, the recombination, mutation and local search take place and an offspring is generated. If the fitness of the offspring is better than the supporter's that took part in the recombination, the offspring replaces the supporter. If the new individual is already present in the population, it is not inserted in it. We adopted a policy of not allowing duplicated individuals to reduce loss of diversity. After the generation is over and all individuals were inserted, the population is restructured. The hierarchy forces the fitness of an individual to be lower than the fitness of the individual just above it in the ternary tree. Following this policy, the higher clusters will have leaders with better fitness than the lower clusters and the best solution will be the leader of the root cluster. The adjustment is made by comparing each individual to the individual just above which it is connected to. If the lower individual becomes better than the upper one, they swap places.

Local search

In contrast to the complex insertion/removal neighborhood search scheme utilized in the TS algorithm, the local search developed for the MA approach is simpler though it makes use of two different neighborhoods. The first neighborhood is defined by the *All-Pairs Interchange* moves. It consists of systematically swapping the alleles in all positions of a given solution. A hill-climbing algorithm can be defined by reference to this neighborhood. Let s be a given solution. Every swap (or move) is confirmed if, and only if, one of the two following conditions apply:

- The move improves the makespan of solution s
- The move does not change the makespan of solution s , but it reduces or keeps constant the completion time of the machines involved in the move.

If an improvement is observed after all the interchanges, the process is repeated until improvement no longer occurs. The resulting solution is a local minimum with respect to the All-Pairs neighborhood. The second neighborhood is based on Insertion moves. It consists of removing an allele from one position and inserting it into another one. The hill-climbing procedure is the same as the All-Pairs scheme. It also finishes in a local minimum with respect to the Insertion neighborhood. In fact, the local search applied is a 3-phase hill-climbing procedure which combines both neighborhoods, as shown next.

Phase 1 - Apply the All-Pairs-based local search to a starting solution.

Phase 2 - Apply the Insertion-based local search to the solution obtained at the end of Phase 1.

Phase 3 - If the solution obtained in Phase 2 is identical to the starting solution, stop. The solution is a local minimum for both neighborhoods. Otherwise, return to Phase 1, with the starting solution updated.

This procedure is very time-consuming, being responsible for more than 90%, in average, of the total processing time. Other hill-climbing procedures have been tested such as executing Phases 1 and 2 only once, or making a single pass in each neighborhood, without guaranteeing that the solution obtained is a local minimum. These alternative approaches have not proved to be effective.

Parameter setting

Parameters were empirically set using a partial enumeration procedure applied to three parameters: size of population, crossover rate and mutation rate. The enumeration procedure that determines the best parameter values is shown below. Table 1 shows the values assigned to the minimum and maximum values of each respective range. They define the `Init_Parameters ( )` function.

Table 1. Range values and step sizes used for parameter setting

size of population	crossover rate (%) step size = 10%		mutation rate (%) step size = 10%	
	<i>min_cross_rate</i>	<i>max_cross_rate</i>	<i>min_mut_rate</i>	<i>max_mut_rate</i>
13, 40 and 121 individuals	0	100	0	100

The parameter setting procedure executes a partial search in the space defined by all possible combinations of parameter values. The experiment was conducted on a sample set of instances, referred as *set_instances* in the pseudo-code. The number of machines, structure and range of due-dates play a negligible role in the parameter determination when compared to the number of jobs, which apparently determines the complexity of the problem. Therefore, for a fixed number of jobs, the procedure chooses five instances at random and uses the average makespan as a comparative measure.

The procedure starts with all parameters fixed at their minimum values according to table 1. Then, for a fixed value of n , the procedure tests all possible sizes of population and store the best value found in *best_value*. In the next loop, the crossover rate varies and the population size and mutation rate remain fixed. When it finishes, the best crossover rate is stored. The last loop deals with the mutation rate setting. This completes the first cycle. The *best_value* found and the best parameters are saved and another cycle begins, now with the initial parameters set as the best ones found in the last cycle. This strategy showed that after three cycles at most, no improvement in the makespan was observed. The CPU time used to run an instance was 2 minutes. Table 2 summarizes the final results for the best parameter settings.

Table 2: Parameter setting results

n	Size of Population	Crossover rate	Mutation rate
20	121	0.2	0.1
40	40	0.2	0.1
60	13	0.3	0
80	13	0.2	0

The best number of individuals found was 121 for problems with 20 jobs, 40 individuals for problems with 40 jobs, and 13 individuals for problems with 60 and 80 jobs. This descending behavior is easily explained by the fact that, with a fixed time span, it is preferable to have few individuals evolving during more generations than to have many individuals and few generations. The mutation should be set at very low rates, never greater than 0.1. The best crossover rates were found between 0.2 and 0.3. These numbers should not be taken strictly, but just as guiding values, since we have not observed a significant sensitivity with respect to them.

4. COMPUTATIONAL RESULTS

The instances used in the computational experiments were randomly generated. The number of jobs was fixed at 20, 40, 60 and 80. The number of machines was fixed at 2, 4, 6 and 8.

Processing times were generated following a discrete uniform distribution $DU(1, 100)$. Setup times were divided into two categories: small setup times – with values in the interval $[1, 10]$ and large setup times – with values in the interval $[1, 100]$. The setup times were also generated according to two possibilities: structured and non-structured. The structured setup times follow the triangular inequality, that is, $s_{ij} \leq s_{ik} + s_{kj}$, $\forall i, j, k \neq i, j$. The non-structured setup times do not follow the triangular inequality, which usually makes the problems harder to solve. We considered 10 replications for each configuration of number of jobs and machines.

The aim is to compare the performance of the three meta-heuristics: Long Tabu (LT), Fast Tabu (FT) and memetic algorithm (MA). For small-sized instances, the methods could be compared to optimal solutions yielded by an exact method based on a dichotomous search (Franca *et al.*, 1996). For those instances in which the exact method was not able to find an optimal solution, a good lower bound provided by the dichotomous search was employed as a baseline for comparisons. However, for some instances the exact method fails to find even an lower bound. In this case, comparisons are made relative to the solutions provided by LT (marked with symbol ‘–’ in the tables). It is worth mentioning that, for the dichotomous search, the factor that determines how hard is the solution process is the number of machines. Hence, the ones with $m = 2$ and 4 are referred as ‘small problems’, and as ‘large problems’ when $m = 6$ and 8. Next, we present four tables, with the results for structured and non-structured problems and setup times in the two intervals of generation. The numbers in tables 3-6 are expressed as the average relative percentage deviation of makespan. The stopping criterion for both tabu search implementations was chosen as the number of iterations ($10n$) with no improvement in the incumbent solution. Two stopping criteria based on CPU time were implemented for MA:

- 2 minutes of CPU time for each configuration (MA_{2m}).
- The same CPU time used for Fast Tabu (MA_{eq}).

The software was programmed in Java JDK 1.2 and run using a PC-Compatible PENTIUM II 266 MHz. As the TS and the MA were programmed in different languages and run on different machines, let us examine this issue further. The Java JDK 1.2 and the C compiler are quite similar in terms of speed, with some advantage for the C compiler. On the other hand, the SUN SPARC 10 is a little slower than a Pentium II 266 MHz.

Each table is divided in two parts: small and large problems. As said before, for the small problems the comparisons are made against optimal solutions or lower bounds. The values are percentage deviations. Consider Table 3, column FT and $m=2$. In this case we have an average deviation of 4.0%. That means the average makespan was 4.0% worse than the respective lower bounds or optimal values.

Table 3. Structured problems, $s_{ij} \in [1,100]$

Small problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 2	1.6	1.0	0.1	1.8
40 / 2	3.8	2.8	2.0	3.0
60 / 2	5.1	3.8	3.2	3.9
80 / 2	5.4	4.6	4.9	4.3
Average	4.0	3.1	2.6	3.3
20 / 4	5.3	4.2	2.0	4.4
40 / 4	6.5	5.9	4.6	8.4
60 / 4	7.0	6.5	7.0	9.1
80 / 4	6.7	6.5	7.9	8.6
Average	6.4	5.8	5.4	7.6
Large problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 6	0.8	-	-1.8	1.3
40 / 6	1.0	-	0.8	4.1
60 / 6	0.4	-	2.0	4.5
80 / 6	0.7	-	4.1	4.2
Average	0.7	-	1.3	3.5
20 / 8	1.4	-	-1.1	1.7
40 / 8	1.0	-	1.3	6.6
60 / 8	0.7	-	1.8	5.3
80 / 8	0.4	-	3.7	4.8
Average	0.9	-	1.4	4.6

Table 4. Non-structured problems, $s_{ij} \in [1,100]$

Small problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 2	2.1	1.5	0.0	3.4
40 / 2	4.6	3.6	3.5	4.4
60 / 2	5.2	4.2	5.0	5.2
80 / 2	5.1	4.2	6.2	4.7
Average	4.3	3.4	3.7	4.4
20 / 4	6.0	4.8	2.8	6.6
40 / 4	6.8	5.9	6.4	8.9
60 / 4	7.2	5.9	8.4	9.9
80 / 4	6.9	5.8	9.2	9.5
Average	6.7	5.6	6.7	8.7
Large problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 6	1.6	-	-2.3	1.1
40 / 6	0.8	-	0.7	3.8
60 / 6	0.7	-	2.9	5.0
80 / 6	0.3	-	3.9	4.5
Average	0.9	-	1.3	3.6
20 / 8	1.0	-	-1.3	0.6
40 / 8	1.1	-	1.0	4.1
60 / 8	0.8	-	3.8	5.2
80 / 8	0.9	-	4.4	5.5
Average	1.0	-	2.0	3.9

For larger problems, the comparison is made against the Long Tabu (the LT column appears empty) and the numbers represent the percent deviation of the respective method. Consider Table 3 again, column FT and $m=6$. In this case we have an average deviation of 0.7%. That means the average makespan was 0.7% worse than the values obtained by the Long Tabu. Occasionally there will appear some negative values considering some large problems. These values indicate results better than the ones obtained by Long Tabu. For structured problems with setups in the interval $[1, 10]$, it was not possible to find optimal or lower bounds for the cases with 4 machines ($m = 4$). In this case, the comparison was also made against the Long Tabu.

The performance of the MA appears to be better when the number of machines is small and when the setup times are small compared to the processing times. The problems with structured setup times also appear to be easier for the MA than the non-structured ones.

Table 5. Structured problems. $s_{ij} \in [1,10]$

Small problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 2	0.6	0.5	0.0	0.5
40 / 2	0.9	0.7	0.3	0.6
60 / 2	0.9	0.7	0.5	0.5
80 / 2	1.1	0.8	0.6	0.5
Average	0.9	0.7	0.4	0.5
20 / 4	0.3	-	-0.7	0.0
40 / 4	0.2	-	-0.5	-0.1
60 / 4	0.3	-	-0.3	-0.1
80 / 4	0.3	-	-0.1	-0.1
Average	0.3	-	-0.4	-0.1
Large problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 6	0.7	-	-0.9	-0.2
40 / 6	0.2	-	-0.3	0.3
60 / 6	0.2	-	-0.1	0.2
80 / 6	0.2	-	-0.1	0.1
Average	0.3	-	-0.4	0.1
20 / 8	1.0	-	-0.5	-0.2
40 / 8	0.2	-	-0.6	0.2
60 / 8	0.1	-	-0.2	0.3
80 / 8	0.1	-	0.1	0.2
Average	0.4	-	-0.3	0.1

Table 6. Non-structured problems. $s_{ij} \in [1,10]$

Small problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 2	0.6	0.4	0.1	0.3
40 / 2	0.7	0.5	0.3	0.3
60 / 2	0.5	0.4	0.3	0.3
80 / 2	0.5	0.2	0.2	0.2
Average	0.6	0.4	0.2	0.3
20 / 4	1.9	1.8	1.0	1.7
40 / 4	1.6	1.4	1.3	1.3
60 / 4	1.1	0.9	0.9	0.9
80 / 4	0.8	0.6	0.8	0.8
Average	1.4	1.2	1.0	1.2
Large problems				
n / m	FT	LT	MA _{2m}	MA _{eq}
20 / 6	0.4	-	-1.2	-0.8
40 / 6	0.2	-	-0.2	0.1
60 / 6	0.2	-	0.1	0.1
80 / 6	0.3	-	0.1	0.3
Average	0.3	-	-0.3	-0.1
20 / 8	1.4	-	-0.7	-0.7
40 / 8	0.7	-	-0.5	0.2
60 / 8	0.3	-	0.1	0.4
80 / 8	0.3	-	0.3	0.5
Average	0.7	-	-0.2	0.1

There are 16 rows labeled ‘Average’ in the tables, and in 12 of them, the MA_{2m} outperforms the FT. The MA_{eq} also outperforms the FT in 9 cases and in 10 cases the MA_{2m} obtains better results than the LT. This last comparison reveals a very good performance, since the Long Tabu runs for much more time than the MA in average. Table 7 shows the average CPU times in seconds consumed by FT and LT to satisfy the stopping criterion. We remark that the performance of LT was achieved at the expense of a large CPU time.

An ANOVA test – two-factor without replication – was executed in order to analyze the influence of the type of instance on the performance of the algorithms and also to analyze the difference between algorithms considering the entire set of instances. The two cases are referred to as ‘instance’ and ‘algorithm’, respectively, in Table 8. The tests were conducted separating the results into four sets, with small and large problems, and the two intervals of setup times. The α -value adopted in all cases was 0.05. For small problems the test indicates that all the algorithms are greatly influenced by the configuration of the instance. In the same way, each algorithm has a considerable different performance from the others. For large problems the results were somewhat different. The ANOVA test indicated that the algorithms

continue to perform very differently from each other, considering the entire set of instances, but the influence of the configuration of the instance is much less significant. For $s_{ij} \in [1, 10]$ the F-value was below the F-critical value and for $s_{ij} \in [1, 100]$ the F-value was just 0.2 above the critical value. The P-values were always very small, except by two values for large problems – 0.72 and 0.037. They reinforce the conclusion that the configuration of the instance has little influence on the performance of the methods, considering the large problems.

Table 7. Fast and Long Tabu CPU times

Setup times	Dimension	Structure	m	FT	LT
$s_{ij} \in [1, 10]$	Small	structured	2	90.7	9887.5
			4	22.3	2097.7
		non-structured	2	180.9	17580.2
			4	34.0	4452.2
	Large	structured	6	14.8	635.8
			8	10.5	297.4
		non-structured	6	26.6	1270.0
			8	19.1	597.2
$s_{ij} \in [1, 100]$	Small	structured	2	112.6	13609.2
			4	31.8	3060.4
		non-structured	2	104.6	9768.2
			4	28.4	3051.3
	Large	structured	6	19.7	949.8
			8	12.9	421.2
		non-structured	6	18.9	875.2
			8	12.9	247.0

Table 8. ANOVA results

Setup times	Size	Data	F-critical	F-value	P-value
$s_{ij} \in [1, 10]$	Small	Instance	1.89	48.47	1.02×10^{-22}
		Algorithm	2.81	30.49	6.48×10^{-11}
	Large	Instance	2.01	0.74	7.20×10^{-1}
		Algorithm	3.31	11.99	1.49×10^{-4}
$s_{ij} \in [1, 100]$	Small	Instance	1.89	18.61	1.76×10^{-14}
		Algorithm	2.81	8.78	1.07×10^{-4}
	Large	Instance	2.01	2.19	3.27×10^{-2}
		Algorithm	3.31	21.01	1.97×10^{-06}

5. CONCLUSIONS

In this paper we have compared three solution techniques for solving a parallel machine scheduling problem with sequence-dependent setup times ($P|sds|C_{max}$). Two of them are based on Tabu Search (Fast Tabu -FT- and Long Tabu -LT-) and the third is a memetic algorithm (MA). The approaches FT and LT embody new neighborhood structures for removal and insertion of jobs responsible for the very near optimal solutions achieved by these methods. The MA approach uses a hierarchically structured population and a local search which employs a combination of the well known swap and insertion neighborhoods. Computational tests were executed using four sets of problems, with large/small and structured/non-structured setup times. For each set of problems we tested problems with 2, 4, 6 and 8 machines and 20, 40, 60 and 80 jobs. For 2 and 4 machines the methods are evaluated by means of comparisons with optimal solutions obtained by a dichotomous search. For larger problems (6 and 8 machines), comparisons are made with respect to a lower bound provided by the dichotomous search or to the solutions found by LT. Results showed a superior performance of the MA when the setup times are small compared to the processing times. The FT algorithm outperformed MA for problems with large setup times and many machines: $m = 6$ and $m = 8$. Also relevant is the procedure proposed to select the best parameters for the MA approach. It is an iterated procedure that executes a partial search in the space of possible combinations of parameters. These ‘optimized’ parameters consequently led to a good performance of the MA. The proposed MA approach can also be applied to other performance measures, such as sum of completion times, average workflow, etc. The adaptation in this case would include modifications just in the fitness function, which should consider the new performance measure. The other parts of the MA remain unchanged. Finally, in order to quantify the relations between algorithms, and the influence of the configuration of the instances in their performance, we conducted a two-factor ANOVA test without replication. The results showed that the performance of the algorithms is greatly influenced by the parameters of the instances.

Acknowledgements

This work was supported by the Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq)-Brazil and the Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP)-Brazil. We would like to thank the Centre des Recherche sur les Transports, Université de Montréal, Canada, for the computational support in executing the tabu search experiments.

REFERENCES

- BAKER, K. R. 1974, *Introduction to Sequencing and Scheduling* (John Wiley & Sons).
- DEARING, P. M., and HENDERSON, R. A., 1984, Assigning looms in a textile weaving operation with changeover limitations, *Production and Inventory Management* 25, 23-31.

- FRANÇA, P. M., GENDREAU, M., LAPORT, G., and MULLER, F., 1996, A tabu search heuristic for the multiprocessor scheduling problem with sequence dependent setup times, *International Journal of Production Economics*, 43, 79-89.
- FRANÇA, P. M., MENDES, A., and MOSCATO, P., 1999, A memetic algorithm for the total tardiness single machine scheduling problem, *European Journal of Operational Research* (to appear).
- BURIOL, L., FRANÇA, P. M., and MOSCATO, P., 1999, Recursive arc insertion: a new local search embedded in a memetic algorithm for the asymmetric traveling salesman problem, submitted to *Journal of Heuristics*.
- FREDERICKSON, G., HECHT, M. S., and KIM, C. E., 1978, Approximation algorithm for some routing problems, *SIAM Journal on Computing* 7, 178 - 193.
- GENDREAU, M., HERTZ, A., and LAPORTE, G., 1992, New insertion and post-optimization procedures for the traveling salesman problem, *Operations Research* 40, 1086-1094.
- GLOVER, F., and LAGUNA, M., 1997, *Tabu Search* (Kluwer).
- GRAHAM, R. L., LAWLER, E. L., LENSTRA, J. K., and RINNOOY KAN, A. H. G., 1979, Optimization and approximation in deterministic sequencing and scheduling: a survey, *Annals of Discrete Mathematics*. 5, 287 - 326.
- MOSCATO, P., 1989, On evolution, search, optimization, genetic algorithms and martial arts: towards memetic algorithms, *Caltech Concurrent Computation Program, C3P Report* 826.
- MOSCATO, P., 1999, Memetic algorithms: a short introduction. In D. Corne, M. Dorigo, and F. Glover (eds.) *New Ideas in Optimization* (McGraw-Hill).
- SUMICHRIST, R., and BAKER, J. R., 1987, Scheduling parallel processors: an integer linear programming based heuristic for minimizing setup time, *International Journal of Production Research* 25, (5), 761 - 771.